机器人小车智能工厂综合项目 实验指导书（合并版）

> 课程代码: EXP007
> 总学时: 32
> 实验数: 4
> 适用对象: 人工智能/机器人工程/智能科学与技术 本科高年级（有 ROS2 与单模块工程基础）
> 制定日期: 2026-09-23
> 模式: human_waiver（源码位于 ROS2 工作空间 `ros2_ws/src/xflz_eai_car1`，未复制到 source-codes/）

目录

1. 实验1：机器人建图与导航
2. 实验2：机器人视觉与抓取
3. 实验3：机器人离线大模型语音交互
4. 实验4：机器人语音控制导航抓取综合应用

实验总览

编号	名称	学时	类型	难度	源码路径
exp01	机器人建图与导航	8h	综合性	⭐⭐⭐	`ros2_ws/src/xflz_eai_car1/slam/car1_slam/` + `navigation/car1_navigation/`
exp02	机器人视觉与抓取	8h	综合性	⭐⭐⭐	`ros2_ws/src/xflz_eai_car1/app/car1_app/` + `arm/car1_arm_controller/`
exp03	机器人离线大模型语音交互	8h	综合性	⭐⭐⭐	`ros2_ws/src/xflz_eai_car1/interaction/car1_speech/` + `car1_llm/`
exp04	机器人语音控制导航抓取综合应用	8h	综合性	⭐⭐⭐	`ros2_ws/src/xflz_eai_car1/app/car1_app/`

学时合计: 8+8+8+8 = **32h**

类型分布: 综合性 4 个

难度分布: ⭐⭐⭐ 4 个

跨实验知识脉络

```
exp01 建图导航（出地图+自主移 动）──► 坐标基准（initial/pick/place_pose）
                                                  |
                                                  ▼
exp02 视觉抓取（检 测红色方块→三维定位→抓/放）
        |
        ▼
exp03 离线语音+大模型意图（唤醒→识别→start 意图）
        |
        ▼
exp04 语音控制导航抓取综合应用（状态机闭环，link 全部）
```

> 核心链路一句话：**先建图导航（exp01）→ 视觉抓红色方块（exp02）→ 离线语音说"出发"（exp03）→ 状态机串成全流程闭环（exp04）。**

实验快速验证命令（完整闭环）

```
# exp01 一键建图（bringup + 雷达 + slam_toolbox + RViz）
ros2 launch car1_slam slam.launch.py
# 遥控建图（另开终端）
ros2 run car1_slam slam_teleop.py
# 保存地图
ros2 run car1_slam map_save.py
# 一键导航（加载地图 + AMCL + Nav2）
ros2 launch car1_navigation navigation.launch.py map:=slam_map

# exp02 视觉抓取前置（先 bringup 再 arm，顺序反了 arm 不响应）
ros2 launch car1_bringup bringup.launch.py
ros2 launch car1_arm_controller arm_controller.launch.py

# exp03 离线语音大模型（唤醒 → 识别 → 意图）
ros2 launch car1_interaction full_interaction.launch.py

# exp04 全栈闭环（导航 + 机械臂 + 相机 + 状态机）
ros2 launch car1_app navigation_grasp_transport.launch.py map:=slam_map
# 触发任务（另开终端）
ros2 service call /navigation_grasp_transport/start std_srvs/srv/Trigger
```

实验1：机器人建图与导航（8h）

> ✅ **源码已核实** 本实验对应仓库内两个已交付功能包：`slam/car1_slam`（SLAM_Toolbox 二维激光建图）与 `navigation/car1_navigation`（Nav2 导航栈）。根目录 = `ros2_ws/src/xflz_eai_car1`。这是"智能工厂综合项目"的第一环：先在真实场地用遥控建出一张栅格地图，再让机器人依赖这张地图实现 AMCL 定位与自主导航，为后续"到指定位置抓红色方块"提供坐标基础。

1.1 关联知识点

本实验对应"智能工厂综合项目"第 1 章"建图与导航"，覆盖以下知识点（ID 遵循 course-spec 分级约定）：

- 1.1.1 综合任务架构 > 概念 > 智能工厂闭环由「建图导航 → 视觉抓取 → 离线语音+大模型意图 → 语音控制闭环」四大环节构成，本实验夯实第一环的坐标与导航基础
- 1.1.2 SLAM 与占据栅格地图 > 原理 > SLAM（同时定位与建图）三要素：感知→定位→构图；栅格地图以 0/255/-1 表示占据/空闲/未知
- 1.1.3 SLAM_Toolbox 建图 > 流程 > `slam.launch.py` 启动 slam_toolbox + 激光雷达 + car1 底盘，`slam.yaml` 配置分辨率/回环/更新策略
- 1.2.1 遥控建图与地图保存 > 流程 > `slam_teleop.py` 遥控移动覆盖环境；`map_save.py` 保存 pgm+yaml 到 `maps/`
- 1.2.2 AMCL 定位 > 原理 > 粒子滤波自适应定位；`localization.launch.py` 启动 map_server + AMCL + lifecycle
- 1.3.1 Nav2 全局/局部规划 > 原理 > ThetaStar 全局规划器 + DWB 局部规划器 + `velocity_smoother` 速度平滑
- 1.3.2 代价地图与导航目标 > 流程 > Local/Global Costmap 膨胀层与激光障碍检测；设定 `pick_pose` / `place_pose` 目标点，`BasicNavigator.goToPose` 自主移动

1.2 实验目标

1. 掌握 SLAM_Toolbox 二维激光建图的流程与参数，能够用遥控器/键盘把智能工厂场地建成可复用的栅格地图 — 独立完成
2. 掌握 AMCL 粒子滤波定位与 Nav2 全局/局部导航调用，理解"建图-定位-规划"三者如何衔接 — 原理理解
3. 在已知地图上设定导航目标点，让机器人自主移动到指定位置（抓取点/放置点），验证导航成功率与精度 — 独立完成

1.3 实验环境

类别	配置
硬件	具身智能机器人小车 C1（STM32F407VET6 主控 + 4 轮麦克纳姆底盘 + 5DOF 机械臂 + COIN-D6 激光雷达 + HP60C 深度相机 + BMI088 IMU）；具身智能实验箱 B2
OS	Ubuntu 22.04 + ROS2 Humble（车载板）；25Hz 激光 / 1Mbps STM32 串口
核心库	rclpy、slam_toolbox、nav2、navigation2、slam_toolbox、tf2、map_server、amcl
依赖包	`car1_bringup`（bringup.launch.py 启动底盘/URDF/EKF/舵机）、`car1_peripherals`（激光雷达）、`car1_slam`、`car1_navigation`
主入口	`ros2 launch car1_slam slam.launch.py`（建图） `ros2 launch car1_navigation navigation.launch.py map:=slam_map`（导航）

1.4 实验难度

- 难度: ⭐ ⭐ ⭐ 综合性

- 重点: 建图 → 定位 → 导航的完整链路，及与上层抓取任务的坐标对接（`initial_pose` / `pick_pose` / `place_pose`）
- 难点: AMCL 初始位姿给的准不准、地图质量好不好，直接决定后续导航与抓取定位是否可靠；`/cmd_vel` 与 `/controller/cmd_vel` 两条速度通道的钳制差异

1.5 实验现象

快速验证流程（全程在车载 C1 板执行）：

1. 启动建图（含 bringup + 激光雷达 + slam_toolbox）：

```
ros2 launch car1_slam slam.launch.py
```

2. 打开 RViz 建图视图，遥控小车遍历场地，观察占据栅格地图逐步生成：

```
ros2 run car1_slam slam_teleop.py
# 或 键盘键位 单独遥控（按 q 退出，r 右手系 / l 左手系，键位在脚本内）
```

3. 地图基本成型后保存：

```
ros2 run car1_slam map_save.py   # 保存到
~/ros2_ws/src/xflz_eai_car1/slam/car1_slam/maps/slam_map.pgm + .yaml
```

4. 关闭建图，改用已知地图启动导航（定位 + Nav2）：

```
ros2 launch car1_navigation navigation.launch.py map:=slam_map
```

5. 在 RViz 中用 `2D Pose Estimate` 设置初始位姿，再用 `2D Goal Pose` 点击目标点，观察小车自主规划路径并移动到位，终端/面板显示 `Goal reached`。

执行现象：

- 建图阶段：RViz 的 Map 图层随小车移动不断变为占据栅格（白色=空闲、黑色=占据、灰色=未知），同时显示激光扫描线（`/scan`）与 EKF 轨迹；遥控转弯过快或撞墙会导致地图拖影/畸变
- 定位阶段：AMCL 粒子从初始位姿附近扩散并收敛，`/amcl_pose` 与 `/particlecloud` 显示红色粒子聚拢到机器人周围（50-100cm 误差内）
- 导航阶段：RViz 显示全局路径（绿线）+ 局部代价地图（滚动窗口 3m×3m）+ 局部轨迹（DWB 采样），小车以 `max_vel_x=0.3` 平滑行驶，到达后绿线消失、状态机置位
- 坐标对接：`/cmd_vel` 被 odom_publisher 钳制到 ±0.2 m/s；Nav2 内部走 `/controller/cmd_vel`（无钳制）保证自身控制速度不受限，见 CLAUDE.md「Velocity Command Routing」

1.6 实验步骤

整体流程图



步骤1：智能工厂综合任务架构认知（阶段一：能运行，30min）

> 📌 本步骤对应源码：`ros2_ws/src/xflz_eai_car1/CLAUDE.md`

知识点讲解：

综合任务闭环（1.1.1）

- **原理**：智能工厂综合项目不是把各模块简单堆叠，而是"语音指令 → 意图识别 → 建图导航 → 视觉抓取 → 运输放置"的一体化闭环。实操中，小车先（在本实验）建出场地地图，再用同一张地图导航到目标位置，随后机器人移动机械臂观察桌面红色方块、检测并抓取、导航到放置点放下。这一环节产出的地图与坐标（`pick_pose`、`place_pose`）是后续所有环节的空间基准，务必保证其质量。

数据流与速度通道（关键工程约束）

- **原理**：底盘速度有两条通道（见 `CLAUDE.md`）：外部 / 应用命令走 `/cmd_vel`（odom_publisher 施加 ±0.2 m/s 安全钳制），Nav2 规划输出走 `/controller/cmd_vel`（无钳制）。`navigation_base.launch.py` 中两处 cmd_vel 的 remap 必须保留，否则 Nav2 速度命令会落回 `/cmd_vel` 被钳制到 0.2 而"走得很慢"。底盘里程计先出 `odom_raw`，经 EKF（robot_localization）融合 IMU 后出 `/odometry/filtered`，Nav2 的 bt_navigator 订阅该话题（`nav2_params.yaml` L61）。

操作步骤：

1. 阅读 `CLAUDE.md` 的 Architecture 与 Launch Commands，梳理五种 launch 的关系（bringup / peripherals / slam / navigation / app）
2. 确认 `.bashrc` 已 source 工作空间，执行 `ros2 launch car1_slam slam.launch.py` 前的依赖服务；用一个终端随时 `ros2 topic list` 观察新增话题

易错点：

- ⚠️ 先启动底盘与传感器（slam.launch 内含 bringup），否则激光 `/scan` 与 `/odometry/filtered` 无数据，slam_toolbox 建图无输入

步骤2：SLAM_Toolbox 建图参数与启动（阶段二：能理解，40min）

> 📌 本步骤对应源码：`slam/car1_slam/launch/slam.launch.py`、`slam/car1_slam/config/slam.yaml`

知识点讲解：

SLAM_Toolbox 配置（1.1.3）

- **原理**：`slam.launch.py` 依次 include bringup（底盘 + URDF + EKF + 舵机）、peripherals（激光雷达），再拉起 `slam_toolbox` 节点并加载 `slam.yaml`。`slam.yaml` 关键参数决定建图质量：
  - `resolution`（栅格分辨率，m/像素）——越小地图越精细，但文件越大、建图计算越重；
  - `map_update_interval`（地图更新时间）—— 越低刷新越频繁；
  - 回环检测（scan matching + loop closure）—— 让机器人回到已探索区域时能把地图"闭环"，纠正累计漂移。
- `slam_teleop.py` 提供遥控建图脚本：按键控制线/角速度，`q` 退出，`r` / `l` 切换刚体/非完整约束；建图时速度要慢、转弯要匀，过快会导致激光帧间位移过大、图匹配失败。

操作步骤：

1. 打开 `slam.yaml`，逐项阅读 `resolution`、`map_update_interval`、`slam_mode`、`scan_subscriber_queue_size` 等参数语义
2. 启动建图：

```
ros2 launch car1_slam slam.launch.py
```

3. 用 `rqt_graph` 观察建图期节点拓扑：`/scan` → slam_toolbox → `/map`；用 `rviz2 -d slam.rviz` 打开建图视图

易错点：

- ⚠️ 建图时不要中途停车太久或原地快速转圈，机器人姿态突变会让 scan-to-scan 匹配失败
- ⚠️ `resolution=0.05` 是平衡精度与体积的常用值；现场场地较小时可适当调大，但后续 Nav2 costmap 也是 0.05，务必一致

步骤3：遥控建图采集环境（阶段一：能运行，50min）

> 📌 本步骤对应源码：`slam/car1_slam/car1_slam/slam_teleop.py`

知识点讲解：

遥控建图要领（1.2.1）

- **原理**：栅格地图由激光在 robot pose 处的投影累加而成，机器人走到哪里、激光扫到哪里，地图才长到哪里。最终任务要求机器人能从起始点（`initial_pose`）导航到抓取点（`pick_pose`），因此场地中**起始区、抓取点桌面、放置区**务必都被建到、且道路通畅无不可达死角。
- **操作步骤**：
    1. 运行 `ros2 run car1_slam slam_teleop.py`，把小车从起始区出发，遍历过道、绕抓取桌一圈、再回到起始区（形成闭环，触发回环校正）
    2. 观察 RViz 中地图黑色区域（障碍）随激光增长，确认没有明显"断层/空鼓"（地图与真实对不上的条带）
    3. 缓慢控制小车回到起点，让激光再次扫过初始位置，尽量让地图闭合

易错点：

- ⚠️ 地图出现"重影/双层墙"→ 多为转角速度太快或激光被遮挡；放慢速度、控制转角 ≤30°/步重新扫
- ⚠️ 建图完成后直接关掉不要急，多停 3-5 秒让 slam_toolbox 提交回环修正

步骤4：地图保存与加载验证（阶段一：能运行，30min）

> 📌 本步骤对应源码：`slam/car1_slam/car1_slam/map_save.py`、`slam/car1_slam/maps/slam_map.yaml`

知识点讲解：

地图保存（1.2.1）

- **原理**：地图以两文件保存：`slam_map.pgm`（灰度图像）与 `slam_map.yaml`（元数据：分辨率、原点、阈值、栅格合法性）。`map_save.py` 调用 nav2 map_saver 写出，供导航阶段的 `map_server` 加载。`slam_map.yaml` 中 `free_thresh` / `occupied_thresh` 决定灰度→占据/空闲判定。

- **操作步骤**：
    1. 保存：`ros2 run car1_slam map_save.py`，确认生成 `maps/slam_map.pgm`（可 `python`/看图软件打开预览）
    2. 阅读 `slam_map.yaml` 确认 `resolution`、`image`、`origin`（负值表示地图原点在图像中心的左下平移）
    3. 用 `ros2 run nav2_map_server map_server ...` 或直接进入导航 launch 验证加载

易错点：

- ⚠️ 保存脚本默认路径在 `slam/car1_slam/maps/`，若改名务必在导航启动时用 `map:=<名字>` 指定一致
- ⚠️ `origin` 是地图左下角相对 `map` 坐标系的偏移，`initial_pose` 需基于 `map` 坐标系给出，二者不能搞混

步骤5：AMCL 定位与初始位姿设置（阶段二：能理解，50min）

> 📌 本步骤对应源码：`navigation/car1_navigation/launch/include/localization.launch.py`、`config/nav2_params.yaml`（amcl 段）

知识点讲解：

AMCL 粒子滤波（1.2.2）

- **原理**：`localization.launch.py` 用 `RewrittenYaml` 把 `yaml_filename`（地图）与 `use_sim_time` 注入同一份 `nav2_params.yaml`，拉起 `map_server`、`amcl`、`lifecycle_manager_localization` 三个节点并自动激活。AMCL 在 `initial_pose`（`nav2_params.yaml` L46-50，本例为起始停车位坐标）附近散布 500-2000 个粒子，靠激光与地图的似然收敛；`initial_particle_pose` 的协方差（L52-54 x/y=0.25、yaw=0.068）决定初始不确定性大小。
- 定位不准的本质是"地图坐标"与"机器人当前世界坐标"没对齐。提供初始位姿时应把粒子布到机器人真实所在处，精度越高收敛越快。

操作步骤：

1. 启动导航（含定位）：`ros2 launch car1_navigation navigation.launch.py map:=slam_map`
2. 在 RViz 用 `2D Pose Estimate` 点击小车真实位置并拖出朝向，设置初始位姿
3. 观察 `/particlecloud` 粒子从"一团"收敛到"一小撮"；用 `ros2 topic echo /amcl_pose` 对比 TF 中 `map→base_footprint` 的一致性
4. 故意把初始位姿摆偏 1m，观察 AMCL 是否仍能收敛（验证鲁棒性）

易错点：

- ⚠️ 定位假设"机器人在已知地图中"——若初始位姿偏差过大（超出粒子覆盖），可能收敛到错误点；保守做法是手动搬到已知位置再设置
- ⚠️ `/scan` 是 Best Effort QoS，RViz 中 LaserScan 的 Reliability 需设为 Best Effort，否则看得到 `/scan` 却不刷新

步骤6：Nav2 全局/局部导航与目标点自主移动（阶段三：能改造，90min）

> 📌 本步骤对应源码：`navigation/car1_navigation/config/nav2_params.yaml`（controller/planner/costmap 段）、`navigation/car1_navigation/launch/navigation.launch.py`

知识点讲解：

Nav2 规划链（1.3.1、1.3.2）

- **原理**：`nav2_params.yaml` 定义了完整规划链：
  - **全局规划器** `planner_server` 用 `nav2_theta_star_planner/ThetaStarPlanner`（L185）——任意角落数/启发式代价权重，在 Global Costmap（静态地图 + 障碍 + 膨胀）上求无碰路径；
  - **局部规划器** `controller_server` 用 `dwb_core::DWBLocalPlanner`（L138）——在 3×3m 滚动 Local Costmap 上按时间采样速度空间（vx=20、vθ=40 样本、sim_time=1.5s），用代价/对齐/转向等多个 critic 加权选最优轨迹；`max_vel_x=0.3`、`max_vel_theta=1.0`；
  - **速度平滑** `velocity_smoother` 对规划输出做限速/限加速度钳制（`max_velocity` / `max_accel`）。
- **行为树导航**：`bt_navigator` 用 `navigate_to_pose_without_backup.xml`（去掉了 Backup 恢复动作，见 L71）加载 NavigateToPose 行为树，封装 ComputePath → FollowPath → 目标判定。

操作步骤：

1. 用 RViz `2D Goal Pose` 在抓取点位置设目标，观察全局路径（ThetaStar）与小车实际跟随（DWB）是否贴合、有无绕路
2. 在路径上临时放一个纸箱，观察 Local Costmap 障碍层把路径"顶开"，小车自动绕障
3. 阅读 `nav2_params.yaml` 的 DWB critic 权重（PathAlign 32 / GoalAlign 24 / PathDist 32 / GoalDist 24），理解各权重对"贴边/居中/冲目标"的影响
4. 用 Python 侧 `nav2_simple_commander` 的 `BasicNavigator` 发起一次 `goToPose` 编程导航（与后续综合实验一致）：

```
from nav2_simple_commander.robot_navigator import BasicNavigator
nav = BasicNavigator()
goal = PoseStamped()   # header.frame_id='map', 设 x/y/yaw 为 pick_pose
nav.goToPose(goal)
while not nav.isTaskComplete():
    print(nav.getFeedback())
```

易错点：

- ⚠️ 小车能够规划但原地不动/爬行 → 检查 `/cmd_vel` 通道钳制问题（见步骤 1 速度通道）
- ⚠️ 全局路径绕大弯 → 调整 Global Costmap 的 `inflation_radius`（L306 为 0.18）或 ThetaStar 的 `w_heuristic_cost`

步骤7：pick/place 坐标对接与导航复测（阶段三：能改造，50min）

> 📌 本步骤对应源码：`app/car1_app/config/navigation_grasp_transport.yaml`（poses 段）

知识点讲解：

任务坐标对接（1.3 延伸）

- **原理**：综合任务用三组地图系坐标驱动导航（见 `navigation_grasp_transport.yaml` L4-9，实测 2026-07-08）：
  - `initial_pose: [-0.059, -0.103, 0.020]` —— 起始/停车区；

    - `pick_pose: [0.756, 0.872, 3.071]` —— 抓取点（`.last` 代表朝向 rad）；
    - `place_pose: [-0.059, -0.103, 0.020]` —— 放置点（与起始点相同）。
- 这三个坐标必须与**本实验建出的地图**一致；若重新建图，需用 `2D Pose Estimate` 重新标定，或在导航稳定后读取 TF 更新配置（如 `navigation_grasp_transport` 的 `set_pick_pose_from_current` 服务）。

操作步骤：

1. 结合步骤 6，用 BasicNavigator 依次编程导航到 `pick_pose`（停车朝向让桌面进入相机视野）与 `place_pose`，各测 3 次记录成功率与到位误差
2. 观察导航到位后小车停车朝向是否便于相机观测桌面（后续 ROTATE_SEARCH 以到位航向为基准）
3. 填写性能基线表（报告必填）：

指标	期望值	实测值
建图分辨率	0.05 m/像素	
全局路径规划耗时	<1s	
导航到位误差	≤6cm（goal_tolerance 0.06）	
前往 pick_pose 成功率	3/3	
前往 place_pose 成功率	3/3	

易错点：

- ⚠️ 重新建图后坐标全部失效——务必同步更新 `navigation_grasp_transport.yaml` 的 poses，否则导航到旧位置

1.7 常见问题

问题	原因	解决方法
建图出现重影/双层墙	转角速度过快或激光遮挡	放慢速度、控制转角 ≤30°/步；建图结束前多停几秒触发回环
导航时小车爬行	Nav2 走回 `/cmd_vel` 被钳制 ±0.2	确认 `navigation_base.launch.py` 两处 cmd_vel remap 未改动（见 CLAUDE.md）
定位收敛到错误位置	初始位姿偏差超出粒子覆盖	手动把机器人放回已知位置，用 `2D Pose Estimate` 重设
RViz 中 `/scan` 不刷新	LaserScan QoS 是 Best Effort	把 RViz LaserScan 的 Reliability 设为 Best Effort
全局路径绕大弯	inflation_radius 过大或启发式权重失衡	调 `global_costmap.inflation_layer.inflation_radius`（0.18）或 ThetaStar `w_heuristic_cost`
map_server 加载失败	map:= 名称与保存名不一致	用 `map:=slam_map` 与 `maps/slam_map.pgm` 严格对应

1.8 扩展实验

扩展方向 1：多点连续导航（智能工厂巡逻）

- 在场地中定义 3-4 个 waypoint，用 `nav2_simple_commander` 的 `goThroughPoses` / `followWaypoints` 编写顺序巡逻节点，验证多目标点调度与连续性

扩展方向 2：导航与抓取联动的坐标预校验

- 在把导航结果交给视觉抓取前，用一个脚本读取 `/amcl_pose`，与 `pick_pose` 做欧氏距离校验，超过容差则报警——作为综合任务（exp04）的可靠性前置

1.9 实验报告要求

评分标准：架构认知 10 分 + SLAM 建图 20 分 + 地图保存加载 10 分 + AMCL 定位 20 分 + Nav2 全局/局部导航 25 分 + 坐标对接与复测 15 分 = 100 分。

报告必含内容：

1. 综合任务闭环框图，标注本实验承担的"坐标与导航"环节
2. 建好的 `slam_map.pgm` 截图与 `slam_map.yaml` 关键参数
3. AMCL 粒子收敛前后 RViz 截图
4. 前往 pick/place 的全局路径 + 到位误差记录表
5. 回答思考题：为什么 Nav2 走 `/controller/cmd_vel` 不走 `/cmd_vel`？初始位姿给错了对后续抓取定位的连锁影响是什么?
6. 实验心得与问题记录

实验2：机器人视觉与抓取（8h）

> ✅ **源码已核实** 本实验核心实现位于 `app/car1_app/car1_app/navigation_grasp_transport.py` 的视觉与抓取模块（`_detect_and_project` / `_count_red_pixels` / `_refresh_joints_sync` / `_handle_detect_and_grasp` / `_handle_call_pick`），配置见 `config/navigation_grasp_transport.yaml`。任务要求：机器人导航到位后，移动机械臂使装在前端的 HP60C 深度相机下视桌面，识别并三维定位红色方块，通过 `arm_controller` 服务完成抓取与放置。

2.1 关联知识点

本实验对应"智能工厂综合项目"第 2 章"视觉与抓取"，覆盖知识点：

- 2.1.1 HSV 颜色空间 > 原理 > HSV（色相/饱和度/明度）比 RGB 对光照更鲁棒；红色色相绕 0/180 回绕，需双阈值区间（lower `[0,100,80]`-`[15,255,255]` 与 lower2 `[160,100,80]`-`[180,255,255]`）
- 2.1.2 形态学去噪与轮廓提取 > 流程 > `cv2.inRange` → 椭圆核 `MORPH_OPEN` / `MORPH_CLOSE` → `findContours`+判面积 → 取质心 `(u,v)`
- 2.2.1 深度反投影 > 原理 > 用相机内参把像素坐标投影为相机三维坐标：$x_{cam}=(u-c_x)z/f_x,\ y_{cam}=(v-c_y)z/f_y$；HP60C 深度为 uint16 毫米，需 `/1000` 转米
- 2.2.2 深度采样鲁棒化 > 流程 > 在物体轮廓掩膜内取有效深度做中位数 + MAD 离群剔除，抗空洞与边缘噪声
- 2.2.3 TF 坐标变换 > 原理 > `camera_depth_optical_frame` → `base_link` 变换把相机坐标转到机器人 base 系，供 `set_pick_target` 使用

- 2.2.4 舵机姿态刷新 > 流程 > `refresh_positions` 同步刷新 bus 舵机实际角度，防止 TF 依据"指令位置"而非"实际位置"导致抓取偏移
- 2.3.1 arm_controller 服务 API > 流程 > `set_pick_target`（只存目标不移动）/ `pick`（执行抓取）/ `place`（执行放置）/ `set_gripper` / `go_home` / `go_work`
- 2.3.2 观察位姿与放置目标整定 > 流程 > `observe_pose`（相机下视桌面）与 `place_target_arm`（放置点相对 base）参数

2.2 实验目标

1. 掌握 HSV 颜色阈值识别红色方块，并用形态学与轮廓处理稳定提取质心 — 独立完成
2. 掌握深度相机反投影 + TF 坐标变换，把图像上的目标换算成 `base_link` 下的三维坐标 — 原理理解
3. 掌握 arm_controller 抓取/放置服务调用，成功抓取红色方块并放到指定位置 — 独立完成

2.3 实验环境

类别	配置
硬件	具身智能机器人小车 C1（含 5DOF 机械臂 + HP60C 深度相机 + bus 舵机）；具身智能实验箱 B2
相机话题	RGB：`/ascamera_hp60c/camera_publisher/rgb0/image`；深度：`.../depth0/image_raw`；内参：`.../depth0/camera_info`
相机帧	`camera_depth_optical_frame`（光学坐标系），机器人基座帧 `base_link`
核心库	cv2、numpy、cv_bridge、tf2_ros、tf2_geometry_msgs
服务 API	`/arm_controller/set_pick_target` (PoseStamped)、`/arm_controller/pick` (Trigger)、`/arm_controller/place` (Trigger)、`/arm_controller/set_gripper` (SetBool)、`/arm_controller/go_home`、`/arm_controller/go_work`、`/servo_controller/refresh_positions` (Trigger)
主入口	`navigation_grasp_transport.py`（`_detect_and_project` / `_handle_detect_and_grasp`）

2.4 实验难度

- 难度: ⭐ ⭐ ⭐ 综合性
- 重点: "像素 → 相机坐标 → 机器人 base 坐标"的整条三维定位链路，及 arm_controller 服务化抓取
- 难点: 相机与 base 之间的 TF 链；bus 舵机"指令/实际"位置不一致导致的定位漂移；深度相机的近盲区（HP60C 约 0.5m）

2.5 实验现象

快速验证流程（综合任务已跑通状态，实机验证前务必先做 PWM 安全检查）：

1. 前置：`ros2 launch car1_bringup bringup.launch.py` + `ros2 launch car1_arm_controller arm_controller.launch.py`（需先 bringup，顺序反了会导致 arm 无响应）
2. 让机器人导航到位，服务端触发抓取（或直接单独测试视觉模块）
3. 观察日志输出 `_detect_and_project` 的三段关键打印：
    - `[DETECT] centroid=(ux,vy) area=... z_cam=... | cam_frame=(x,y,z)`
    - `[TF] cam_frame=... → base_link=[xv,yv,zv]`
    - `Published set_pick_target: [x, y, z]`
4. 终端看到 `FINAL target` 后，机械臂自动下探沿相机反投影方向抓取，随后夹爪闭合提起
5. 调用 `/arm_controller/place` 放置，夹爪张开、机械臂缩回工作位

预期效果描述：

- **HSV 检测**：红色方块在掩膜中呈现为高亮连通区域，阈值过滤掉非红背景；面积小于 100px 的噪声被过滤
- **深度反投影**：`_detect_and_project` 打印的 `z_cam` 是轮廓内深度中位数（米），`cam_frame` 为相机系坐标，`base_link` 为经 TF 变换后的抓取目标

- **抓取精度**：`camera_depth_optical_frame→base_link` 变换正确 + `refresh_positions` 刷新实际角度时，抓取目标与方块中心误差在 1-2cm 内；若不刷新舵机实际角度，x 方向可能偏移约 2cm 而抓空
- **放置结果**：机械臂把方块放到 `place_target_arm=[0.26, 0.0, 0.08, 3.14159]`（base 系正前方 26cm、高 8cm、夹爪朝下），随后 `go_work` 回工作位、夹爪闭合进入待命

2.6 实验步骤

整体流程图



步骤1：HSV 颜色阈值识别红色方块（阶段二：能理解，40min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `_count_red_pixels`（L952-958）、`_detect_and_project`（L987-993）；配置 `navigation_grasp_transport.yaml` L58-63

知识点讲解：

HSV 与红色双阈值（2.1.1）

- **原理**：HSV 把颜色拆成色相 H（0-180）、饱和度 S、明度 V，天然把"颜色"与"光照/亮度"解耦。红色特殊在它横跨色相环的 0° 与 180°（品红/红/橙连续），单区间覆盖不全，因此用两个区间**取或**：
  - `lower=[0,100,80]` ~ `upper=[15,255,255]`（偏红的暖色带）
  - `lower2=[160,100,80]` ~ `upper2=[180,255,255]`（偏品红的高色相端）
    阈值越低越容易把肤色、橙色误识别为红；阈值越高越抗干扰但可能漏检暗红。

操作步骤：

1. 读 `navigation_grasp_transport.yaml` L58-63，确认两套阈值与默认值
2. 单独跑一个临时脚本，用一个静态拍摄的红色方块图验证掩膜：

```
import cv2, numpy as np
img = cv2.imread('red_cube.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
m = cv2.bitwise_or(
    cv2.inRange(hsv, np.array([0,100,80]), np.array([15,255,255])),
    cv2.inRange(hsv, np.array([160,100,80]), np.array([180,255,255])))
print('red_pixels =', int(m.sum()//255))
```

   （浅色方块/加灯后如漏检，可适当降 S/V 下界或扩 H 上界）
3. 记录光照变化（开灯/关灯）下 red_pixels 数量变化，体会 HSV 对比 RGB 的稳健性

易错点：

- ⚠️ OpenCV 的 HSV 用 `H∈[0,180]`，不是教科书 0-360；写 `[160,100,80]` 不要误会
- ⚠️ 红色双阈值必须 `bitwise_or`，缺 2 区间会在 0/180 边界漏检

步骤2：形态学去噪与轮廓质心提取（阶段一：能运行，40min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` L992-1010

知识点讲解：

形态学与质心（2.1.2）

- **原理**：`getStructuringElement(MORPH_ELLIPSE,(5,5))` 生成椭圆核；`MORPH_OPEN`（先腐蚀后膨胀）去掉小椒盐噪声，`MORPH_CLOSE`（先膨胀后腐蚀）填补方块内部小空洞。随后 `findContours(RETR_EXTERNAL)` 取外轮廓，`contourArea` 过滤面积 <100 的噪点，取最大轮廓，用图像矩 `M['m10']/M['m00']`, `M['m01']/M['m00']` 求质心像素 `(u,v)`。

操作步骤：

1. 在步骤 1 脚本基础上补形态开/闭 + 取最大轮廓 + 质心，复现在红框标注质心
2. 记录按摩托后 `mask` 的噪声减少、方块区域更完整
3. 思考并回答：为什么用最大轮廓而不是所有轮廓？桌面反光误检到较大非红区域时如何规避？

易错点：

- ⚠️ 质心是 `(u,v)=(x,y)` 像素坐标，颜色是 BGR 下反投影输入，注意后续公式里 u 对应列、v 对应行

步骤3：深度采样与反投影（阶段二：能理解，60min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `_detect_and_project` L1012-1049；相机内参来自 CameraInfo

知识点讲解：

深度采样鲁棒化与反投影（2.2.1、2.2.2）

- **原理**：HP60C 深度图是 uint16 毫米。`_detect_and_project` 用轮廓掩膜选有效深度（`valid=(contour_mask>0)&(depth>0)`），取中位数并做 MAD 离群剔除：`inliers = depths[|depths-med| <= 2.5*MAD]`，再用中位数作为该点深度 `z_raw`。`depth` 是整数类型时 `/1000` 转米得 `z_cam`。最后反投影：

$$x_{cam} = \frac{(u - c_x) \cdot z_{cam}}{f_x}, \quad y_{cam} = \frac{(v - c_y) \cdot z_{cam}}{f_y}$$

  其中 (c_x,c_y) 为主点、f_x,f_y 为焦距（来自 `camera_info_topic`）。

- **为什么不用单点深度**：单点可能踩到空洞/条带；用轮廓内中位数 + MAD 剔除能同时抗"深度空洞"与"前景干扰"，得到稳定一致的深度。

操作步骤：

1. 读 `_detect_and_project` L1012-1049，标注深度采样、单位换算、反投影三处
2. 用相机原地观测方块 10 次，记录每次 `z_cam` / `cam_frame`，统计离散度（应 <2cm，若大则是深度噪声或方块太小采样不足）
3. 回答问题：`MIN_DEPTH_SAMPLES=30`（L1019）的意义；深度近盲区（<0.5m）为何会让反投影失效或跳变

易错点：

- ⚠️ 深度单位是毫米，必须 `/1000`；忘了会在 z 上差 1000 倍

- ⚠️ `camera_info` 对应 depth 相机，务必订阅 `depth0/camera_info`

步骤4：refresh_positions 刷新实际关节角（阶段三：能改造，40min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `_refresh_joints_sync`（L904-950）

知识点讲解：

bus 舵机"指令 vs 实际"位置（2.2.4）

- **原理**：`/joint_states` 默认发布的是**指令（commanded）关节角**，而 TF 树（`camera_depth_optical_frame→base_link`）由 robot_state_publisher 依据 joint_states 生成。若机械臂还有残差未到位，指令位置 ≠ 物理实际位置，则 TF 会给出错误的相机位姿，导致 detected 目标的 base 坐标整体偏移（实测 x 偏约 2cm 就足以抓空）。`_refresh_joints_sync`（L904）同步调用 `/servo_controller/refresh_positions`，强制 bus 控制器回读实际角度并更新 TF，超时 1.5s 则退化用旧 TF。
- **为什么放在 TF 查询前**：只有在读取实际角度之后再去查 `camera_depth_optical_frame→base_link`，变换矩阵才基于真实的机械臂姿态。

操作步骤：

1. 读 `_refresh_joints_sync` L904-950，理解 `call_async` 后 1.5s 内轮询 `future.done()` 的同步等待
2. 分别注解掉/保留 refresh 调用各跑一次抓取，对比 base 坐标打印与抓取成败，体会该步的"关键性"
3. 记录：不 refresh 时 `base_link` 目标与 refresh 时的差值（理论上接近一个固定的 TF 位姿误差）

易错点：

- ⚠️ 该步骤是"抓空"的头号隐性原因——务必保留在 TF 查询前
- ⚠️ 机械臂仍在运动时 refresh 读到的仍是过渡位姿，因此上层会等 `observe_settle_sec=5.0` 让机械臂稳定后再检测（见 exp04）

步骤5：TF 坐标变换到 base_link（阶段二：能理解，50min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` L1056-1085；`do_transform_point`（tf2_geometry_msgs）

知识点讲解：

像素→base 坐标（2.2.3）

- **原理**：相机光学系得到的是相机坐标 $(x{cam},y{cam},z_{cam})$，但机械臂/导航都在 `base_link` 系工作。`tf_buffer.lookup_transform(base_link, camera_depth_optical_frame, Time())` 取变换（含平移+旋转），构造 `PointStamped` 在相机系，用 `do_transform_point(point_optical, tf)` 变换到 base：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{base} = R_{cam}^{base} \cdot \begin{bmatrix} x_{cam} \\ y_{cam} \\ z_{cam} \end{bmatrix} + t_{cam}^{base}$$

  得到的 `(x_base,y_base,z_base)` 即方块相对机器人 collinear 的三维位置，直接喂给 `set_pick_target`。

操作步骤：

1. 读 `_detect_and_project` L1056-1085，确认 TF 查询、`PointStamped` 帧设置与 do_transform 链路
2. 用 `ros2 run tf2_ros tf2_echo camera_depth_optical_frame base_link` 观察当前变换（平移旋转）
3. 验证：把方块放桌面，对比打印的 `[TF]` 行 `(x_base,y_base,z_base)` 与方块在 base 系的大致位置（应为正前方、桌面高度附近）

易错点：

- ⚠️ `header.frame_id` 必须设为 `camera_depth_optical_frame`，否则变换坐标系错误
- ⚠️ TF 查找超时 0.5s；机械臂抖动期间 TF 更新临时中断是正常的，重试即可

步骤6：set_pick_target + pick 服务调用（阶段三：能改造，50min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `_handle_detect_and_grasp`（L660-747）、`_handle_call_pick`（L749-762）

知识点讲解：

"先存目标、下一拍再抓"（2.3.1）

- **原理**：`/arm_controller/set_pick_target`（PoseStamped）只把抓取目标**存进 arm_controller**、不立即动作；真正执行是 `/arm_controller/pick`（Trigger）。设计上二者**分在两个状态（DETECT_AND_GRASP → CALL_PICK）而非同一 tick 连续调用**：给 arm_controller 的订阅回调留一个执行周期去消费 `set_pick_target`，再调用 `pick` 时才能读到刚写入的目标。`pitch=π` 表示夹爪朝下（ik_analytical 约定 0=上、π=下）。
- `_handle_detect_and_grasp` 里先**连续重复检测 N 次（`detection_repeat_count=5`）取最后一次成功结果**——机械臂到观察位后前 ~1s 仍在微调，单次瞬态读数会偏，连续检测取末次能让位姿稳定后再定目标（模拟 visual_grasp 的 WAIT_CONFIRMATION）。

操作步骤：

1. 读 `_handle_detect_and_grasp` L660-747，标注"重复检测取末次"与"发布 set_pick_target"两段
2. 读 `_handle_call_pick` L749-762，理解它如何 `service_is_ready()` 后 `call_async` 并立刻转入 WAIT_GRASP
3. 用 `dry_run: true` 先跑一趟，确认目标发布正确但机械臂不动（安全验证投影与定位链路）
4. 关闭 dry_run，观察夹爪闭合、机械臂按目标下探抓取

易错点：

- ⚠️ 若在同一个 tick 里"发布目标再调 pick"，可能 pick 读到旧目标而抓错位置——这是 CALL_PICK 独立状态的动机
- ⚠️ `dry_run` 下只验证到发布目标为止，务必先 dry_run 再上 real pick，防误触

步骤7：place 放置与回工作位（阶段三：能改造，50min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `_handle_place_object`（L793-900）

知识点讲解：

多阶段放置序列（2.3.2）

- **原理**：放置不是一次服务调用：
    1. 发布放置目标 `place_target_arm=[0.26,0,0.08,π]` 到 `set_pick_target`（不移动）；
    2. 调用 `/arm_controller/place`——arm_controller 内部完成"升到 pre_grasp 高度→下探→张开夹爪释放→上缩"；
    3. 调用 `/arm_controller/go_work` 把机械臂移回工作观察位，为下次导航/抓取准备；
    4. 调用 `/arm_controller/set_gripper(False)` 夹爪闭合进入待命。
    `_handle_place_object` 用多个 future（`_place_future` / `_work_future` / `_gripper_future`）逐个等待，前三者任一非 None 时不再重发 step1，避免"抓-放-抓-放"无限循环。

操作步骤：

1. 读 `_handle_place_object` L793-900，梳理 place → go_work → set_gripper 三步与 future 编排
2. 观察放置点：方块放置位置是否与 `place_target_arm` 吻合；若偏离，微调该参数
3. 全程观察日志 `Place succeeded` → `go_work done` → `Gripper closed. Task complete -> DONE`

易错点：

- ⚠️ 三个 future 的状态合计决定是否会重发 step1——改动时别破坏"三态判断"逻辑
- ⚠️ go_work 后再关夹爪，是为了"运输途中夹爪不夹物也不松乱"，符合现场待命习惯

2.7 常见问题

问题	原因	解决方法
检测不到红方块	HSV 阈值不适配当前光照/方块颜色	调 `target_color_hsv_*` 阈值；确认双区间 OR
`z_cam` 抖动大于 >2cm	深度空洞或轮廓含非方块区域	增大 `MIN_DEPTH_SAMPLES`、加强 MAD 剔除；重拍深度验证
抓空	TF 用了指令关节角未刷新	正确放置 `_refresh_joints_sync()` 在 TF 查询前
抓偏 2cm 左右（x 方向）	机械臂仍在微调就定了目标	调大 `observe_settle_sec`，或重开 `detection_repeat_count` 取末次
pick 抓到旧位置	发布目标后立刻 pick	确保走 CALL_PICK 独立状态（隔一个 tick）
实机抓取可能堵转	PWM 异常或机械干涉	实机前先跑 `test_servo_direct.py` 验证 PWM，遵循项目安全红线
base 坐标明显不对	camera_info/内参或帧设置错误	订阅 `depth0/camera_info`；`header.frame_id=camera_depth_optical_frame`

2.8 扩展实验

扩展方向 1：多颜色/多目标分拣

- 扩展 HSV 到蓝/绿多区间，定义"颜色→放置槽"映射，让机器人在同一桌面对多种颜色方块做分类抓放，贴近真实产线分拣

扩展方向 2：目标丢视野恢复

- 观察 `ROTATE_SEARCH`（exp04 引入）在一组搜索角内遍历后仍找不到目标的处理（`All search angles exhausted` → FAILED），试试增加/加密搜索角序列提升容错

2.9 实验报告要求

评分标准：HSV 阈值调优 15 分 + 形态学/轮廓 15 分 + 深度采样与反投影 25 分 + TF 变换与 refresh 25 分 + 抓取/放置服务调用 20 分 = 100 分。

报告必含内容：

1. 视觉定位链路图（像素→深度→相机反投影→TF→base），标注每步源码与行号
2. 3 次抓取的 `base_link` 目标坐标打印与最终误差记录
3. "refresh vs 不 refresh"的 base 坐标/抓取结果对比
4. `place_target_arm` 整定前后放置位置说明
5. 回答思考题：为什么用中位数+深度采样而非单点深度？为什么 pick 要独立状态与 set_pick_target 分开拍?
6. 实验心得与问题记录

实验3：机器人离线大模型语音交互（8h）

> ✅ **源码已核实** 本实验对应 `interaction/car1_speech`（离线唤醒/识别/合成）与 `interaction/car1_llm`（离线大模型意图识别）两功能包。语音链路完全离线：sherpa-onnx 的 KWS（唤醒词）+ ASR（识别）+ 本地 TTS（合成），加上 RKLLM 内嵌跑 Qwen 大模型做意图分类，全程不依赖公网。

3.1 关联知识点

本实验对应"智能工厂综合项目"第 3 章"离线大模型语音交互"，覆盖知识点：

- 3.1.1 离线语音链路架构 > 概念 > 唤醒→识别→意图→回复 的模块化流水线：KWS 唤醒 → ASR 语音转文字 → 大模型意图分类 → TTS 播报
- 3.1.2 sherpa-onnx KWS 唤醒 > 原理 > zipformer_kws 模型实时检测唤醒词（"小飞小飞"），sensitivity 调灵敏度（越低越灵但易误触发）
- 3.1.3 sherpa-onnx 离线 ASR > 流程 > zipformer_onnx 模型 greedy_search 将语音转文字；hotword 热词纠约定制词
- 3.1.4 GTCRN 神经网络降噪 > 原理 > 流式语音增强（onnx 模型），仅对 KWS 唤醒生效，抗风扇/电机/空调稳态噪声
- 3.2.1 本地 TTS 合成 > 流程 > tts_node 离线合成回复语音与提示音，支持播报打断
- 3.3.1 RKLLM 离线大模型意图识别 > 原理 > rkllm（Qwen3.5-0.8B 量化模型）在 NPU 上做意图分类，命中 `start` 意图 → 调用任务启动服务
- 3.3.2 运行态唤醒闸门 > 流程 > 任务运行中 `running=true` 暂停唤醒，任务结束恢复，防任务中误唤醒打断

3.2 实验目标

1. 掌握 sherpa-onnx 的离线唤醒（KWS）与识别（ASR）部署与参数调优 — 独立完成
2. 掌握本地 TTS 语音合成与播报打断 — 独立完成
3. 掌握 RKLLM 内嵌大模型（Qwen3.5-0.8B）的离线意图识别，并把"出发/开始任务/开始干活"等人类指令分类为 `start` 意图 — 原理理解

3.3 实验环境

类别	配置
硬件	具身智能机器人小车 C1（板载瑞芯微 RK3576，6TOPS NPU）；具身智能实验箱 B2（六麦阵列/麦克风）
OS	Ubuntu 22.04 + ROS2 Humble（板载 ros2_venv，含 sherpa-onnx / rknnlite / rkllm）
语音模型	`/home/iflytek/models/zipformer_kws`（KWS 唤醒，zipformer_kws） `/home/iflytek/models/zipformer_onnx`（ASR 识别） `/home/iflytek/models/ns/gtcrn_simple.onnx`（GTCRN 降噪）
大模型	`/home/iflytek/models/rkllm/Qwen3.5-0.8B_w4a16_g128_rk3576.rkllm`（RKLLM 量化意图模型）
触发服务	`start_service: /mission_orchestrator/start`（LLM 命中 start 意图后调用）
状态话题	`mission_state_topic: /car1_brics_task/state`（running 字段：true 暂停唤醒 / false 恢复）
主入口	`ros2 launch car1_interaction full_interaction.launch.py` （内含 `speech.launch.py`：wake_word_node + tts_node；`llm.launch.py`：大模型相关节点）

3.4 实验难度

- 难度: ⭐ ⭐ ⭐ 综合性
- 重点: 本地化离线语音 + 边缘大模型意图识别的完整部署，理解"唤醒-识别-意图"三级门槛
- 难点: 唤醒灵敏度与误触发的平衡；汽车风扇/电机噪声下的识别鲁棒性；RKLLM 意图提示词（含 ASR 常见误识别词）的配置驱动

3.5 实验现象

快速验证流程（全程板载离线执行）：

1. 启动完整交互：`ros2 launch car1_interaction full_interaction.launch.py`
2. 唤醒：对麦克风说"小飞小飞"，板端播放"在呢"（提示音 `prompt_zainei.wav` 或 TTS 合成），进入听指令状态
3. 指令：说"出发"/"开始任务"/"开始干活"，ASR 转文字后送入大模型意图分类，命中 `start` 意图，随后触发任务启动服务（如在综合任务里就启动建图导航抓取闭环）

4. 无关闲聊：说"今天天气如何"，大模型回复闲聊语（不触发任务动作）
5. 运行闸门：任务运行中（`/car1_brics_task/state` 中 running=true）再说"小飞小飞"无效，任务完成恢复唤醒

预期效果描述：

- **唤醒**：`wake_word_node` 在灵敏度 `sensitivity=0.25` 下对"小飞小飞"稳定唤醒；唤醒词命中即播提示音，避免用户不知是否已唤醒
- **识别+意图**：ASR 把"出发吧/开始干活"等口语转为文字，大模型按 `intent_hints`（`wake_word.yaml` L22-26 的 start 组）把动词命中的都归类为 `start`，从而鲁棒触发任务
- **降噪**：风声/风扇噪声下仍能唤醒；GTCRN 仅对 KWS 生效，ASR 走原始音频（降噪会损害 ASR）
- **完全离线**：全程无外网调用，符合工厂车间内网/无网部署要求，延迟本地可控

3.6 实验步骤

整体流程图



步骤1：离线语音链路架构认知（阶段二：能理解，30min）

> 📌 本步骤对应源码：`interaction/car1_interaction/launch/full_interaction.launch.py`

知识点讲解：

语音链路模块化（3.1.1）

- **原理**：`full_interaction.launch.py` 把 `car1_speech`（语音）和 `car1_llm`（意图）两个 launch **include 进来一起起**，形成"麦克风 → 唤醒/识别 → 大模型 → TTS/扬声器"链路。`car1_speech` 的 `speech.launch.py` 只起 `wake_word_node` 与 `tts_node` 两个节点——注意真实 ASR 已**内含在 wake_word_node 内部**（唤醒后接着就识别命令），旧独立 asr_node 是 stub 假识别已从 launch 移除。
- 设计要点是**三级门槛**：先 KWS 唤醒（低计算、常开），唤醒后短窗口才做 ASR（省算力），再送大模型分类意图——避免大模型常开推理造成功耗与发热。

操作步骤：

1. 读 `full_interaction.launch.py` 与 `speech.launch.py`，画出节点/话题拓扑（wake_word_node、tts_node、LLM 相关节点）
2. 确认 `wake_word_node` 内含"唤醒→识别"的状态机（IDLE 监听 KWS → 命中唤醒 → 播提示音 + 进入 ASR 采集 → 出文字 → 回 IDLE）

易错点：

- ⚠️ 别用旧 asr_node（stub）；真实识别在 wake_word_node 内部，避免误接旧话题污染结果

步骤2：KWS 唤醒词与灵敏度（阶段一：能运行，50min）

> 📌 本步骤对应源码：`car1_speech/config/wake_word.yaml`
> (wake_word/sensitivity/kws_model_dir/keywords_file)

知识点讲解：

sherpa-onnx 唤醒（3.1.2）

- **原理**：`wake_word: "小飞小飞"` 用 sherpa-onnx 的 `zipformer_kws` 模型做实时关键词检测，`sensitivity: 0.25` 是置信度门限——数值越小越灵敏（更易唤醒，也更多误触发），越大越钝（更不易误唤醒）。`keywords_file` 为空时用 `car1_interaction/tools/keywords.txt`（含 小飞小飞/小飞同学/你好小飞 等热词）的 pinyin。唤醒后 `prompt_wav`（`prompt_zainei.wav`）播"在呢"提示音提示继续说指令。

操作步骤：

1. 读 `wake_word.yaml`，确认唤醒词、`sensitivity`、模型路径
2. 在安静环境对麦克风说"小飞小飞"，观察唤醒命中与提示音
3. 逐一调节 `sensitivity`（0.1/0.25/0.5），记录"唤醒率 vs 误触发率"：

sensitivity	唤醒率（10 次）	误触发（10 分钟）
0.1		
0.25（默认）		
0.5		

4. 答问：现场有风扇/电机时如何取舍？为什么要"唤醒后才能说指令"而不是一直开着识别？

易错点：

- ⚠️ 唤醒词是四字"小飞小飞"，两字或多音词识别不稳；改词需同步改 `wake_word` 与 `keywords_file`
- ⚠️ `sensitivity` 太低会频繁被环境声触发，指令还没说就进入 ASR 短窗

步骤3：离线 ASR 与热词（阶段二：能理解，50min）

> 📌 本步骤对应源码：`car1_speech/config/wake_word.yaml`（asr_model_dir）；`car1_speech/tools/*.py`（asr_test/offline_stt）

知识点讲解：

sherpa-onnx 识别（3.1.3）

- **原理**：`asr_model_dir` 指向 `zipformer_onnx` 模型，`greedy_search` 解码做无热词的流式识别。ASR 在唤醒后的短窗口收集语音，输出文字。因为现场指令词汇有限，常用**热词/意图提示词**（`intent_hints`）兜底：即使识别出形近（如"出 发"被识别成带空格），提示词也能帮助意图分类匹配。

操作步骤：

1. 唤醒后依次说"出发""开始任务""开始干活"，查看 ASR 输出的文字（日志），记录形近误识别（如"出 发"）
2. 在 `wake_word.yaml` 的 `intent_hints.start.hints` 里补充你发现的新误识别词，实现"配置驱动"改词不改代码
3. 对比有无热词配置时 `start` 意图的命中率

易错点：

- ⚠️ 识别结果是意图分类的输入，误识别词若未列入 hints 会导致 `start` 漏触发——务必在实验中发现词就补

进配置

步骤4：GTCRN 神经网络降噪（阶段三：能改造，40min）

> 📌 本步骤对应源码：`car1_speech/config/wake_word.yaml`（onnx_denose / onnx_denoise_model）

知识点讲解：

降噪为何只作用 KWS（3.1.4）

- **原理**：`onnx_denoise: true` 启用 `gtcrn_simple.onnx` 流式语音增强，采集线程只收原始音频，`_handle_idle(KWS)` 阶段先降噪再喂 KWS——去稳态背景噪声（风扇/电机/空调）让唤醒更灵敏。**ASR 走原始音频不降噪**：因为降噪可能把说话人语音细节一并削掉，反而更难识别。这是"降噪只帮唤醒、不帮识别"的工程取舍。

操作步骤：

1. 打开 `onnx_denoise: false`，在风机旁测试唤醒率
2. 打开 `onnx_denoise: true`，`onnx_denoise_threads: 2`，重测唤醒率对比，记录降噪前后差异
3. 验证：降噪开关不影响 ASR 识别表现（走原始音频）

易错点：

- ⚠️ 不要为提升识别而把降噪套到 ASR 上——通常反而变差（特征被削）

步骤5：本地 TTS 语音合成（阶段一：能运行，40min）

> 📌 本步骤对应源码：`car1_speech/car1_speech/tts_node.py`、`config/tts.yaml`、`config/prompt_zainei.wav`

知识点讲解：

离线合成与播报（3.2.1）

- **原理**：`tts_node`（`speech.launch.py` 注入 tts.yaml）监听 `/interaction/tts/speak` 话题（String），离线合成语音播放；`prompt_zainei.wav` 是预置提示音（也可 TTS 合成"在呢"）。TTS 支持"播报打断"：新的 speak 请求到达时停止当前播放播新的，保证回复紧跟前一次交互。

操作步骤：

1. 发布一条测试播报：`ros2 topic pub /interaction/tts/speak std_msgs/msg/String "data: '好的，我开始执行任务'" -1`
2. 连续发两条不同音频，观察后一条是否打断前一条
3. 对比用 `prompt_wav=prompt_zainei.wav` 与换空串时唤醒提示音的表现

易错点：

- ⚠️ 音频设备索引 `audio_device_index: -1` 自动检测 ES8388；换声卡需重设

步骤6：RKLLM 离线大模型意图识别（阶段二：能理解，60min）

> 📌 本步骤对应源码：`car1_speech/config/wake_word.yaml`（llm / llm_server 段）；`car1_llm/launch/llm.launch.py`

知识点讲解：

RKLLM 边缘意图（3.3.1）

- **原理**：`llm_server` 段配置内嵌启动一个 flask_server，用 RKLLM 加载 `Qwen3.5-0.8B_w4a16_g128_rk3576.rkllm` 量化模型（4bit 权重/128 group size，专为 RK3576 NPU），`lib_path=/usr/lib/librkllmrt.so`，`venv_python=/home/iflytek/ros2_venv/bin/python3`，服务地址 `http://127.0.0.1:8080`。意图分类由提示词驱动：把 ASR 文字交给大模型，参照 `intent_hints` 判断是否属于 `start`（出发/开始/执行任务……）。命中后 `start_service: /mission_orchestrator/start` 被调用，进入任务执行。
- 因为它是"分类而非自由对话"，提示词把**标准词 + ASR 常见误识别词**都 fold 进 `intent_hints`，新发现误识别词直接补配置不用改代码。

操作步骤：

1. 读 `wake_word.yaml` 的 `llm_server` 段，确认模型路径/平台/服务端口
2. 验证意图服务可访问：`curl http://127.0.0.1:8080/...` 或运行 `car1_llm/tools/llm_test.py` 喂入"出发去做任务"，观察返回 `start`
3. 喂入闲聊"今天天气如何"，观察不命中 `start`
4. 在 `intent_hints.start.hints` 新增你发现的误识别词，重启服务验证命中

易错点：

- ⚠️ 大模型是"意图分类"工具，不要期望它对任意指令都动手；它只把命中 start 的任务词挑出来
- ⚠️ `wake_word.yaml` 里 `llm.server_url` / `llm_server.*` 端口被占时复用现有服务，确保端口不与其它 demo 冲突

步骤7：意图驱动与运行态唤醒闸门（阶段三：能改造，60min）

> 📌 本步骤对应源码：`car1_speech/config/wake_word.yaml`（start_service / mission_state_topic）

知识点讲解：

从"识别"到"任务"的桥接（3.3.2）

- **原理**：LLM 命中 `start` 意图后调用 `start_service: /mission_orchestrator/start`，把语音意图接到上层任务编排。同时订阅 `mission_state_topic: /car1_brics_task/state`：当其中 `running=true`（任务正在执行）时**暂停唤醒**，避免任务中途被"小飞小飞"打断；任务完成 `running=false` 恢复唤醒。这就完成了"语音"与"机器人执行"之间的双向节流。

操作步骤：

1. 把语音链路接到一个轻量任务（先接一个打印/亮灯的 Trigger 服务或直连下层任务，见 exp04 的端到端）
2. 说"出发执行任务"，观察 `start` 触发任务启动、`/car1_brics_task/state` 变 running=true
3. 任务运行中再说"小飞小飞"，确认不再唤醒（闸门生效）；任务完成后再唤醒恢复

4. 填性能表：从唤醒到任务启动的总延迟（应本地可控、毫秒级推理 + 传输）

易错点：

- ⚠️ 闸门依赖 `mission_state_topic` 的 running 字段由任务侧正确发布——综合任务状态机需同步维护该话题（见 exp04）

3.7 常见问题

问题	原因	解决方法
唤醒词不响应	sensitivity 太高或环境噪声大	调低 `sensitivity`（0.25→0.1），开 GTCRN 降噪
频繁误唤醒	sensitivity 太低	调高；确保 hotword 准确
"出发"未触发任务	ASR 误识别词未列入 hints	往 `intent_hints.start.hints` 补词（配置驱动）
闲聊也触发 start	提示词意图边界太宽	收紧 hints，只保留明确任务动词
任务中还能被唤醒打断	闸门未生效	检查任务侧是否发布 `/car1_brics_task/state` 且 running=true
大模型服务起不来	端口占用或模型路径错	复用已有服务；核对 rkllm 模型路径与 lib_path
播报不打断	TTS 打断逻辑被跳过	确认 tts_node 订阅 `/interaction/tts/speak` 且实现打断

3.8 扩展实验

扩展方向 1：多意图扩展（停车/避障/供料）

- 在 `intent_hints` 增加 `stop`（停止）、`pause`（暂停）等意图，映射到 `/arm_controller/go_home` 或 `/cmd_vel` 零速话题，形成更完整的产线指令集

扩展方向 2：唤醒语料鲁棒性评测

- 采集多人/不同音量/不同距离下"小飞小飞"唤醒样本，绘制 sensitivity 的 ROC，确定本机最优工作点

3.9 实验报告要求

评分标准：链路架构 10 分 + KWS 唤醒灵敏度调优 20 分 + ASR 与热词 20 分 + GTCRN 降噪对比 10 分 + TTS 合成/打断 10 分 + RKLLM 意图识别 20 分 + 闸门与延迟 10 分 = 100 分。

报告必含内容：

1. 离线语音链路框图（麦克风→KWS→ASR→大模型→TTS），标注各节点与话题
2. sensitivity 灵敏度对比表（3.6 步骤 2 表）
3. 误识别词补充记录与 `intent_hints` 配置片段
4. GTCRN 开/关唤醒率对比
5. 回答思考题：为什么降噪只作用 KWS 不作用 ASR？为什么用大模型做意图分类而不是十几个 if/else？运行闸

门的作用?

6. 实验心得与问题记录

实验4：机器人语音控制导航抓取综合应用（8h）

> ✅ **源码已核实** 本实验是"智能工厂综合项目"的收官闭环，把实验 1（建图导航）、实验 2（视觉抓取）、实验 3（离线语音大模型）通过 `navigation_grasp_transport.py` 的**任务状态机**串成"语音指令 → 意图识别 → 建图导航 → 视觉抓取 → 运输放置"的一体化流程。任务入口为 `/navigation_grasp_transport/start`（`nav2_params/skills.yaml` 中 transport 技能即调用它），一键 launch 见 `navigation_grasp_transport.launch.py`。

4.1 关联知识点

本实验对应"智能工厂综合项目"第 4 章"语音控制导航抓取综合应用"，覆盖知识点：

- 4.1.1 任务状态机设计 > 原理 > `State` 枚举（IDLE→SET_INITIAL_POSE→WAIT_LOCALIZATION→NAV_TO_PICK→MOVE_TO_OBSERVE→WAIT_ARM_STABLE→ROTATE_SEARCH→DETECT_AND_GRASP→CALL_PICK→WAIT_GRASP→NAV_TO_PLACE→PLACE_OBJECT→DONE），单状态机调度所有模块
- 4.1.2 回调组与并发 > 原理 > ReentrantCallbackGroup（服务/客户端）与 MutuallyExclusiveCallbackGroup（定时器）+ MultiThreadedExecutor(4线程)
- 4.2.1 原地旋转搜索 > 流程 > `ROTATE_SEARCH` 从 `map→base_footprint` 读航向，对 `search_angles_deg`（±15/30/45°）逐一以 P 控制器转到位、检测红色像素
- 4.2.2 检测→抓取的分拍调用 > 流程 > 发布 `set_pick_target` 后独立进 `CALL_PICK` 再调 pick，保证 arm 回调已消费目标
- 4.2.3 等待与超时管理 > 流程 > localization/detection/grasp/nav 四类超时与 FAILED 兜底、DONE 5s 复位 IDLE
- 4.3.1 语音→任务桥接 > 流程 > 实验 3 的 `start` 意图调用 `/mission_orchestrator/start` → navigation_grasp_transport 的 `~/start`；任务运行中 `Running` 暂停唤醒
- 4.3.2 端到端联调与故障分析 > 综合 > 定位漂移 / 检不到目标 / 抓取失败 的排查方法

4.2 实验目标

1. 掌握以状态机为核心的复杂任务编排，理解"事件驱动 + 超时兜底 + 状态隔离" — 原理理解
2. 掌握旋转搜索、深度定位、抓取/放置、导航运输多模块在一个状态机下的协同 — 独立完成
3. 掌握"语音指令 → 意图识别 → 建图导航 → 视觉抓取 → 运输放置"全流程闭环，实现一次完整抓取任务 — 独立完成

4.3 实验环境

类别	配置
硬件	具身智能机器人小车 C1（5DOF 机械臂 + HP60C RGB-D + COIN-D6 激光 + RK3576 NPU）；具身智能实验箱 B2

类别	配置
一键启动	`ros2 launch car1_app navigation_grasp_transport.launch.py map:=slam_map`
任务入口	服务 `/navigation_grasp_transport/start`（Trigger）
运行话题	`~/status`（String，当前状态名）、`~/error_reason`（String）
依赖	实验 1 地图 + 实验 2 视觉/抓取 + 实验 3 语音意图
技能映射	`car1_llm/config/skills.yaml`：`transport` 技能 → `/navigation_grasp_transport/start`

4.4 实验难度

- 难度: ⭐ ⭐ ⭐ 综合性（capstone）
- 重点: 复杂任务状态机设计与全链路集成；用日志/话题观测定位各模块偶发故障
- 难点: 多模块时序耦合（导航到位时机、机械臂微调、检测取末次、抓取超时）；语音闸门与任务状态的同步

4.5 实验现象

快速验证流程:

1. 一键启动全栈（含 bringup/导航/机械臂/相机/主节点，Timing 依 launch 注释 t+0/8/10/15s）：

```
ros2 launch car1_app navigation_grasp_transport.launch.py map:=slam_map
```

2. 另开终端触发任务：

```
ros2 service call /navigation_grasp_transport/start std_srvs/srv/Trigger
```

3. 观察日志状态流转，应依次出现：

```
STATE: IDLE -> SET_INITIAL_POSE
Sent initial pose (...) to AMCL
STATE: SET_INITIAL_POSE -> WAIT_LOCALIZATION
Localization settle done
STATE: WAIT_LOCALIZATION -> NAV_TO_PICK
Nav2 goToPose[pick]: (0.756, 0.872, 3.071)
Nav2 [pick] SUCCEEDED
STATE: NAV_TO_PICK -> MOVE_TO_OBSERVE
Moving arm to observation pose: x=0.20 y=0.00 z=0.22 pitch=180.0°
STATE: MOVE_TO_OBSERVE -> WAIT_ARM_STABLE
Arm stable, camera ready
STATE: WAIT_ARM_STABLE -> ROTATE_SEARCH
Search angle +0.0 deg (idx 0): red_pixels=...
STATE: ROTATE_SEARCH -> DETECT_AND_GRASP
Detect [1/5]: (...)
```

```
...
FINAL target (from last of N detections): (x, y, z)
Published set_pick_target: [..]
STATE: DETECT_AND_GRASP -> CALL_PICK
Calling /arm_controller/pick...
STATE: CALL_PICK -> WAIT_GRASP
Pick succeeded: ...
STATE: WAIT_GRASP -> NAV_TO_PLACE
Nav2 goToPose[place]: (-0.059, -0.103, 0.020)
Nav2 [place] SUCCEEDED
STATE: NAV_TO_PLACE -> PLACE_OBJECT
Place succeeded / go_work done / Gripper closed. Task complete -> DONE
STATE: PLACE_OBJECT -> DONE   (5s 后 -> IDLE)
```

4. 语音闭环：实验 3 链路就绪后，说"出发执行任务"，大模型命中 `start` → 触发本任务 → 复现步骤 3 全流程，且任务运行中唤醒被闸门暂停

预期效果描述：

- **导航**：小车从起始区 AMCL 收敛后，ThetaStar 全局 + DWB 局部规划到 `pick_pose`（桌面前方），停车朝向使相机可观测桌面
- **观察**：机械臂移到 `observe_pose=[0.20,0,0.22,π]`（相机下视桌面），等 `observe_settle_sec=5` 稳定
- **搜索**：`ROTATE_SEARCH` 依次转 ±15/30/45°，每角停 1.5s 检测红色像素，首个超过 `red_pixel_threshold=1000` 即切入抓取
- **抓取**：连续重复检测 5 次取末次得到稳定 `base_link` 目标，发布 `set_pick_target`，下一拍调 `pick`，夹爪抓到红色方块
- **运输放置**：导航回 `place_pose`，place 服务在 `place_target_arm=[0.26,0,0.08,π]` 释放方块，go_work 回位、夹爪闭合 → DONE → 5s 后回 IDLE 可再次触发
- **语音**：运行中 `Running` 状态暂停唤醒，任务结束恢复，实现"一次语音、全程自动"

4.6 实验步骤

整体流程图



步骤1：任务状态机整体认知（阶段二：能理解，50min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `State` 枚举（L57-71）、`_enter` / `_tick_body` / `_elapsed_in_state`（L421-467）

知识点讲解：

单一状态机编排（4.1.1）

- **原理**：任务用**一个状态枚举**（13 个状态）把所有模块纳入单一调度：`IDLE→SET_INITIAL_POSE→WAIT_LOCALIZATION→NAV_TO_PICK→MOVE_TO_OBSERVE→WAIT_ARM_STABLE→ROTATE_SEARCH→DETECT_AND_GRASP→CALL_PICK→WAIT_GRASP→NAV_TO_PLACE→PLACE_OBJECT→DONE`（`FAILED` 兜底）。

`_tick`（L421）以 0.1s 定时器周期执行，按当前状态派发到对应 handler；`_enter`（L383）记录进入时刻并按状态做一次性清理（如进 ROTATE_SEARCH 时记录基准航向、进 DETECT 时清空目标）；`_elapsed_in_state`（L418）做状态内超时判定。**关键设计**：每个状态只做一件确定的事，状态之间通过 `_enter()` 转移，状态内用 `_elapsed_in_state()` 守卫超时，避免状态卡死。

操作步骤：

1. 读 `State` 枚举与 `_tick_body`，画出状态转移表（当前态 → 触发条件 → 下一态）
2. 重点标注三处特殊拆分：`CALL_PICK` 独立于 `DETECT_AND_GRASP`；`ROTATE_SEARCH` 与 `DETECT_AND_GRASP` 分开；`PLACE_OBJECT` 内含多段 future
3. 用 `ros2 topic echo /navigation_grasp_transport/status` 实时观测状态名随任务推进

易错点：

- ⚠️ 状态转移都经 `_enter()` 完成初始化，不要直接改 `self.state`，否则状态内清理条件不生效

步骤2：回调组与并发模型（阶段三：能改造，40min）

> 📌 本步骤对应源码：`navigation_grasp_transport.py` `__init__` L200-209、`main` L1119-1131

知识点讲解：

并发与阻塞（4.1.2）

- **原理**：节点启动 0.1s 定时器、多个订阅/服务客户端。为避免"阻塞性同步等待"（如 `_detect_and_project` 内 `time.sleep` 与 `_refresh_joints_sync` 的 1.5s 同步轮询）卡住其它回调，代码用：
  - `ReentrantCallbackGroup`（`_srv_cbg` / `_client_cbg`）给服务与客户端——允许并发回调；
  - `MutuallyExclusiveCallbackGroup`（`_timer_cbg`）给 tick 定时器——同一时刻只跑一个 tick，天然防并发改状态；
  - `MultiThreadedExecutor(num_threads=4)` 提供多线程，让 timer 与 service callback 并行。
- 若全用默认单线程、或把服务回调与 timer 放同一互斥组，同步等待会阻塞 tick 导致卡死。

操作步骤：

1. 读 `__init__` L200-209 与 `main` L1119-1131，理解三组回调组与 4 线程 executor 的关系
2. 思考题：把 `tick_timer` 改成 Reentrant 会引入什么风险？（并发 tick 可能同时改状态）
3. （可选）把线程数降到 1 复测，观察是否出现与服务回调互等

易错点：

- ⚠️ 状态机字段（self.state 等）由 timer 单线程访问，服务回调只触发 `_enter` 而不直接改字段并发——改动时保持此隔离

步骤3：导航与初始位姿状态（阶段一：能运行，60min）

> 📌 本步骤对应源码：`_handle_set_initial_pose` L471-490、`_handle_wait_localization` L492-497、`_handle_navigate` L538-602

知识点讲解：

初始位姿与连线导航（4.2.1 前置）

- **原理**：
  - `SET_INITIAL_POSE`：构造 `PoseWithCovarianceStamped` 发到 `/initialpose`，把 `initial_pose=[-0.059,-0.103,0.020]` 给 AMCL；协方差设 `x/y=0.25、yaw=0.068`，表示"信任初值但允许 AMCL 微调"。
  - `WAIT_LOCALIZATION`：等 `localization_settle_sec=3` 后确保 nav2 `BasicNavigator` 就绪再进导航。
  - `NAV_TO_PICK/NAV_TO_PLACE`：`_handle_navigate(target,next,tag)` 用 `BasicNavigator.goToPose` 发起，`_nav_started` 防重复发起，轮询 `isTaskComplete / getResult`；失败时**重试一次**（`_nav_retry_used`）再 FAILED；`nav_timeout_sec=300` 超时兜底。

操作步骤：

1. 单独跑一次只到 NAV_TO_PICK 的流程，确认 AMCL 收敛与导航目标 `(0.756,0.872,3.071)` 命中
2. 故意在路径上放障，观察多目标重试与超时 FAILED 表现
3. 用 `ros2 topic echo /initialpose` 验证初始位姿/协方差已发布

易错点：

- ⚠️ 初始位姿误差过大会让 AMCL 收敛到错误点，后续 ROTATE_SEARCH 基准航向 `_search_base_yaw` 也错——先手工 `2D Pose Estimate` 校正再跑任务

步骤4：原地旋转搜索（阶段二：能理解，60min）

> 📌 本步骤对应源码：`_handle_rotate_search` L604-658、`_get_current_yaw` L1089-1096

知识点讲解：

闭环旋转搜索（4.2.1）

- **原理**：机械臂到位后相机可能没正对桌面，`ROTATE_SEARCH` 让小车原地转搜：
  - 进状态时记录基准航向 `_search_base_yaw=_get_current_yaw()`（读 `map→base_footprint` TF 的 yaw）；
  - 对 `search_angles_deg=[0,±15,±30,±45]` 逐角计算目标 `target_yaw=_wrap_pi(base+angle)`；
  - 用 P 控制器闭环：`err=wrap_pi(target-current)`，`tw.angular.z=clamp(1.5*err, ±search_speed)` 发 `/cmd_vel`，`|err|≤search_yaw_tol=0.05` 才停；
  - 停定后 `search_settle_sec=1.5` 让图像稳定，再 `_count_red_pixels` 判红，`≥red_threshold=1000` 则进抓取，否则下一角；**所有角度耗尽**仍未见目标 → FAILED。
- 出主意：`search_speed=0.4`（rad/s）小于 `/cmd_vel` 上限 ±1.0，且靠角度误差驱动，故不会超速。

操作步骤：

1. 在机械臂观察位让小车偏离桌面，触发搜索，观察每角的 `current_search_angle` 话题与旋转到位
2. 把方块放在相机视野外某方向，验证多角搜索能转到"看见"并切换 DETECT
3. 记录每角 `red_pixels` 的变化曲线，理解阈值 1000 的设定依据

易错点：

- ⚠️ `_search_base_yaw` 是进状态那一刻读的基准；若导航到位后还被人推动，基准即失真
- ⚠️ 旋转中别停导航/碰撞脚边，防止 TF 基准漂移

步骤5：检测-抓取-超时管理（阶段三：能改造，70min）

> 📌 本步骤对应源码：`_handle_detect_and_grasp` L660-747、`_handle_call_pick` L749-762、`_handle_wait_grasp` L764-791

知识点讲解：

重复检测与分拍（4.2.2、4.2.3）

- **原理**（承接实验 2）：`DETECT_AND_GRASP` 连续重复检测 `detection_repeat_count=5` 次、间隔 0.3s，**取最后一次**成功结果做目标（机械臂微调期瞬态不稳）；需至少 2 次成功才可信。随后发布 `set_pick_target` 并**独立进** `CALL_PICK`（下一 tick 才调 pick），保证 arm 订阅回调已消费目标。`WAIT_GRASP` 轮询 pick future，`grasp_timeout_sec=60` 兜底；`detection_timeout_sec=30` 兜底检测。
- 所有"外部等待"（pick/place/go_work/gripper 的 future、导航 isTaskComplete）都以**非阻塞轮询 + 超时**处理，不阻塞 executor。

操作步骤：

1. 回顾实验 2 视觉链路；在本实验中用一个 `dry_run: true` 先验证 `FINAL target` 正确且机械臂不动
2. 触发一次真实抓取，观察 `<DETECT_AND_GRASP>` 打印的 5 次 Detect 与末次 target
3. 把 `detection_repeat_count` 改成 1 复测，对比目标稳定性（应看到 x 偏移隐患），理解"取末次"的价值
4. 人为把 `grasp_timeout` 调小，验证 FAILED 兜底（抓不到时不僵死）

易错点：

- ⚠️ 抓不到目标时不要反复触发——先看 `~/error_reason` 与 `[TF]` / `[DETECT]` 日志定位是"没检到"还是"检到但偏移"
- ⚠️ 紫色边框：`FINAL target` 与 `Published set_pick_target` 用的是最后检测结果，日志里前面的 Detect 只是过程，别被误导

步骤6：运输放置与状态复位（阶段二：能理解，60min）

> 📌 本步骤对应源码：`_handle_place_object` L793-900、`_tick_body` DONE/FAILED 分支 L461-467

知识点讲解：

多段放置与复位（4.2.3）

- **原理**：`PLACE_OBJECT` 用三个 future 编排多段动作：
    1. 发 `place_target_arm=[0.26,0,0.08,π]` 到 `set_pick_target`（只存不移动）；
    2. 等 0.3s 后调 `/arm_controller/place`（下探→开夹→释放→上缩），`_place_future` 等待；
    3. place 成功后调 `/arm_controller/go_work`（回工作位），`_work_future` 等待；
    4. 回位后调 `set_gripper(False)`（闭合待命），`_gripper_future` 等待 → `_enter(DONE)`。

三个 future 的"不全 None"判定（L806-807）防止末阶段重复触发 step1 造成"抓-放-抓-放"。`DONE` 保持 5s 后自动回 `IDLE`（可再次 `~/start`）；`FAILED` 也 5s 后回 IDLE。

操作步骤：

1. 观察放置：方块落在 `place_target_arm` 附近，机械臂回工作位、夹爪闭合
2. 复测第二趟：任务完成回 IDLE 后再次 `ros2 service call /navigation_grasp_transport/start`，验证可重复执行
3. 记录 3 趟的 "鼠标→放" 成功率与放置偏差

易错点：

- ⚠️ 改了 place 段 future 逻辑时，务必保持"三态合计"防重复触发
- ⚠️ 连续跑任务前确认方块/场地复位，避免重复抓同一物失败

步骤7：语音闭环联调（阶段三：能改造，100min）

> 📌 本步骤对应源码：`car1_llm/config/skills.yaml`（transport 技能）、实验 3 的 wake_word.yaml；`navigation_grasp_transport.launch.py`

知识点讲解：

语音→任务闭环、闸门同步（4.3.1、4.3.2）

- **原理**：闭环分两线——**执行线**：语音唤醒 → ASR → RKLLM 大模型意图，命中 `start` 调用 `mission_orchestrator/start`；**任务线**：navigation_grasp_transport 提供 `~/start` 供上层触发，运行时把 `Running` 写到 `mission_state_topic` 的 `running=true` 暂停唤醒，任务结束恢复。`skills.yaml` 里已声明 `transport` 技能 → `/navigation_grasp_transport/start`，便于 LLM 侧直接按技能调用。
- 联调要点是**两条时序对齐**：语音触发的那一刻，导航/机械臂/相机都已由 `navigation_grasp_transport.launch.py` 的 TimerAction 就绪（t+0 nav、+8 arm、+10 camera、+15 主节点）。

操作步骤：

1. 分两条 main 栈起步：终端 1 起 `navigation_grasp_transport.launch.py`；终端 2 起 `full_interaction.launch.py`（语音）
2. 确认 `/navigation_grasp_transport/start` 与 `mission_orchestrator/start` 的关系（本课程以主节点 `~/start` 为任务入口；mission_orchestrator 为整场编排壳）
3. 完整一次：说"出发执行任务" → 意图 `start` → 触发任务 → 复现步骤 3-6 全流程 → 任务运行中再唤醒被闸门拦截 → 结束恢复唤醒
4. 故障演练：故意把初始位姿摆偏诱发"检不到目标"或"抓空"，用 `~/error_reason` + 日志定位并修复

易错点：

- ⚠️ 语音与任务串起来后，"导航还没就绪就被触发"的时序问题要靠 15s 延迟与内部 wait 兜住；实测失败多源于初始位姿与地图坐标不对齐
- ⚠️ `mission_state_topic` 的 `running` 必须由任务侧及时置位/复位，否则唤醒闸门失效或过早失效

4.7 常见问题

问题	原因	解决方法
启动后任务不响应触发	时序未就绪或服务未 up	等 ≥15s；`ros2 service list` 确认真有 `/navigation_grasp_transport/start`
导航到位搜不到目标	车位姿/搜索角未覆盖目标	校正初始位姿；加密/加宽 `search_angles_deg`；调 `red_pixel_threshold`
抓取 MISS	机械臂微调期定目标	保持 `detection_repeat_count=5` 取末次；调大 `observe_settle_sec`
语音触发不了任务	意图未命中 start 或服务未转	补 `intent_hints.start.hints` 误识别词；核对 launch 时序
任务中能被唤醒打断	闸门未置位	检查任务侧发布 `mission_state_topic` running=true
状态卡住不进下一步	状态内超时缺失或 future 未完成	查 `_elapsed_in_state` 与 future.done；看 `~/status` 卡在哪态
place 反复抓-放	place 段 future 三态判定被破坏	恢复 `_place/_work/_gripper_future` 合计判空逻辑

4.8 扩展实验

扩展方向 1：多轮/多台车调度

- 在 `mission_orchestrator` 层维护任务队列，支持"连续两趟抓不同目标点"，用状态话题 `queue_len` 让语音播报进度，验证编排层对单机多任务的调度

扩展方向 2：异常恢复（重试升级）

- 给 `WAIT_GRASP` 失败增加"重定位后重试"分支，替代直接 FAILED；评估重试对成功率与安全的影响

4.9 实验报告要求

评分标准：状态机设计与状态转移 20 分 + 回调组/并发 10 分 + 旋转搜索 15 分 + 检测-抓取-超时 20 分 + 运输放置与复位 15 分 + 语音闭环联调 20 分 = 100 分。

报告必含内容：

1. 完整状态转移表，标注触发条件与超时
2. 一次成功任务的日志时间线（导航→观察→搜索→抓取→放置→复位）
3. 语音闭环一次完整操作记录（唤醒词/指令/意图/任务状态）
4. 故障演练记录（现象/定位/修复）
5. 回答思考题：为什么状态机用一个 tick 定时器而非事件回调堆叠？CALL_PICK 独立状态解决了什么？运行闸门为什么按 running 字段暂停唤醒？
6. 综合心得：从单模块到全闭环，最大的集成难点是什么？如何改进？

更新日志

版本	日期	变更
v1.0	2026-09-23	合并版定稿（由 `_parts/experiment-{1..4}.md` 合并，补齐课程信息头、目录、实验总览、跨实验知识脉络与快速验证命令；各实验内容与 course-spec/源码保持原样，源码均已核实）